

RUNBOOK NGÀY 1 — SPRINT 1: DỰNG ARCHITECTURE & BLAZOR SERVER + MUDBLAZOR POC

- **Mục tiêu ngày:** Dựng Solution 4-tier .NET 8, cài đặt dependencies, cấu hình Blazor Server + MudBlazor và triển khai Proof of Concept (PoC) component tương tác thành công.
- **Thời lượng dự kiến:** 120 phút
- **Output bắt buộc:** Solution build success (**0 errors, 0 warnings**) + Demo Blazor tương tác mượt mà tại <https://localhost:5001/demo-blazor>.

1. TỔNG QUAN HỆ THỐNG & METADATA

- **Khung công nghệ (Tech Stack):** .NET 8 Blazor Server + MudBlazor 7.x + ApexCharts.Blazor 3.x + PostgreSQL 16
- **Hạ tầng truyền thông Realtime:** SignalR WebSocket (Persistent Circuit tích hợp mặc định trong Blazor Server)
- **Kiến trúc (Architecture):** 4-Tier Clean Architecture Pattern
 - Crm.Domain: Entities, Value Objects, Enums, Domain Interfaces, Domain Exceptions
 - Crm.Data: EF Core 8, ApplicationDbContext, Configurations, Migrations, Repositories
 - Crm.Business: Services, DTOs, Business Rules, FluentValidation
 - Crm.Jobs: Hangfire Recurring Jobs, Background Workers
 - Crm.Web: Presentation Layer (Blazor Server Components, MudBlazor UI, Layouts, Hubs)
 - Crm.Tests: xUnit, Moq, FluentAssertions Testing Layer

PHASE 1 — KIỂM TRA MÔI TRƯỜNG (10 phút)

Mở PowerShell / Terminal và chạy các lệnh kiểm tra phiên bản công cụ:

```
dotnet --version  
# Kỳ vọng: 8.0.xxx
```

```
psql --version  
# Kỳ vọng: psql (PostgreSQL) 16.x
```

```
git --version  
# Kỳ vọng: git version 2.x.x
```

Lưu ý: Nếu máy trạm chưa cài đặt đủ, hãy tải từ các trang chính thức:

- [.NET 8.0 SDK](#)
- [PostgreSQL 16](#)
- [Git SCM](#)

Khởi tạo thư mục làm việc

```
# Chọn thư mục làm việc (điều chỉnh đường dẫn phù hợp với OS của bạn)
cd D:\Projects
mkdir CrmSolution
cd CrmSolution
```

PHASE 2 — DỰNG SOLUTION 4-TIER (30 phút)

Bước 1: Tạo Solution và các Project thành phần

Chạy khối lệnh sau để thiết lập cấu trúc solution tiêu chuẩn:

```
# 1. Tạo Solution
dotnet new sln -n CrmSolution

# 2. Tạo 4 classlib cho các tầng lõi
dotnet new classlib -o src/Crm.Domain -f net8.0
dotnet new classlib -o src/Crm.Data -f net8.0
dotnet new classlib -o src/Crm.Business -f net8.0
dotnet new classlib -o src/Crm.Jobs -f net8.0

# 3. Tạo Project Blazor Server Web App
# Template blazor với cờ --interactivity Server và --all-interactive
# tự động thiết lập Interactive Server Render Mode toàn cục và cấu hình Components/ thay vì
Pages/
dotnet new blazor -o src/Crm.Web -f net8.0 --interactivity Server --all-interactive

# 4. Tạo Project Unit Test
dotnet new xunit -o tests/Crm.Tests -f net8.0

# 5. Dọn dẹp các file Class1.cs mặc định không sử dụng
rm src/Crm.Domain/Class1.cs
rm src/Crm.Data/Class1.cs
rm src/Crm.Business/Class1.cs
rm src/Crm.Jobs/Class1.cs
```

```
# 6. Đăng ký tất cả projects vào Solution
dotnet sln add src/Crm.Domain/Crm.Domain.csproj
dotnet sln add src/Crm.Data/Crm.Data.csproj
dotnet sln add src/Crm.Business/Crm.Business.csproj
dotnet sln add src/Crm.Jobs/Crm.Jobs.csproj
dotnet sln add src/Crm.Web/Crm.Web.csproj
dotnet sln add tests/Crm.Tests/Crm.Tests.csproj
```

Bước 2: Bố trí Reference giữa các tầng (Dependency Rules)

Thiết lập quan hệ phụ thuộc tuân thủ nguyên tắc Clean Architecture:

```
# Data phụ thuộc vào Domain
dotnet add src/Crm.Data reference src/Crm.Domain

# Business phụ thuộc vào Data và Domain
dotnet add src/Crm.Business reference src/Crm.Data
dotnet add src/Crm.Business reference src/Crm.Domain

# Jobs phụ thuộc vào Business
dotnet add src/Crm.Jobs reference src/Crm.Business

# Web phụ thuộc vào Business và Jobs
dotnet add src/Crm.Web reference src/Crm.Business
dotnet add src/Crm.Web reference src/Crm.Jobs

# Tests phụ thuộc vào Business
dotnet add tests/Crm.Tests reference src/Crm.Business
```

Bước 3: Build xác nhận cấu trúc

```
dotnet build
```

Kỳ vọng: Build succeeded. 0 Warning(s). 0 Error(s).

PHASE 3 — CÀI ĐẶT PACKAGES DỰ ÁN (20 phút)

1. Tầng Data (ORM & Database Driver)

```
cd src/Crm.Data
dotnet add package Microsoft.EntityFrameworkCore --version 8.0.10
dotnet add package Microsoft.EntityFrameworkCore.Design --version 8.0.10
dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL --version 8.0.10
cd ../..
```

2. Tầng Business (PDF Engine, Excel, MailKit, Validation, Security)

```
cd src/Crm.Business
dotnet add package QuestPDF --version 2024.10.3
dotnet add package ClosedXML --version 0.104.1
dotnet add package MailKit --version 4.8.0
dotnet add package FluentValidation --version 11.10.0
dotnet add package BCrypt.Net-Next --version 4.0.3
cd ../..
```

3. Tầng Jobs (Background Worker)

```
cd src/Crm.Jobs
dotnet add package Hangfire.AspNetCore --version 1.8.17
dotnet add package Hangfire.PostgreSql --version 1.20.10
cd ../..
```

4. Tầng Web (Blazor UI Components, ApexCharts, SignalR, Identity & Logging)

```
cd src/Crm.Web
dotnet add package MudBlazor --version 7.15.0
dotnet add package Blazor-ApexCharts --version 3.0.0
dotnet add package Microsoft.AspNetCore.SignalR.Client --version 8.0.10
dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore --version 8.0.10
dotnet add package Serilog.AspNetCore --version 8.0.3
dotnet add package Serilog.Sinks.PostgreSQL --version 4.3.0
cd ../..
```

5. Verify Package Restore & Build

```
dotnet restore
dotnet build
```

Kỳ vọng: Toàn bộ NuGet packages được restore và build thành công không xung đột phiên bản.

PHASE 4 — DỰNG BLAZOR SERVER PROOF-OF-CONCEPT (50 phút)

Bước 1: Khởi tạo cấu trúc thư mục Web

Chuyển vào thư mục src/Crm.Web và bổ sung các thư mục theo kiến trúc thành phần:

```
cd src/Crm.Web
mkdir Components/Pages/Demo
mkdir Components/Shared
mkdir wwwroot/js
cd ../../
```

Cấu trúc phân bổ thư mục tại src/Crm.Web:

```
src/Crm.Web/
├── Components/
│   ├── App.razor           # Host HTML gốc
│   ├── Routes.razor       # Bộ định tuyến Router
│   ├── _Imports.razor     # Global namespaces dùng chung
│   ├── Layout/
│   │   ├── MainLayout.razor # Master Layout chứa MudThemeProvider
│   │   └── NavMenu.razor    # Sidebar điều hướng
│   ├── Pages/
│   │   └── Demo/
│   │       └── Index.razor   # Màn hình Demo PoC (@page "/demo-blazor")
│   └── Shared/
│       ├── MessageComponent.razor # Component con: Thông báo MudAlert
│       ├── CounterComponent.razor  # Component con: Huy hiệu MudChip
│       └── SearchComponent.razor   # Component con: Danh sách MudList
└── wwwroot/
    ├── css/
    └── js/
```

└─ Program.cs

Bước 2: Cấu hình Program.cs

Mở file src/Crm.Web/Program.cs và thay thế toàn bộ nội dung:

```
using MudBlazor.Services;

var builder = WebApplication.CreateBuilder(args);

// 1. Cấu hình Blazor Server với Interactive Server Components
builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();

// 2. Đăng ký MudBlazor UI Services (Dialog, Snackbar, Theme)
builder.Services.AddMudServices();

// 3. Đăng ký ApexCharts.Blazor
builder.Services.AddApexCharts();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error", createScopeForErrors: true);
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseAntiforgery();

// 4. Map Razor Components với Interactive Server Render Mode
app.MapRazorComponents<Crm.Web.Components.App>()
    .AddInteractiveServerRenderMode();

app.Run();
```

Bước 3: Cấu hình Components/_Imports.razor

Cập nhật src/Crm.Web/Components/_Imports.razor để nạp sẵn namespace cho toàn bộ components:

```
@using System.Net.Http
@using System.Net.Http.Json
@using Microsoft.AspNetCore.Components.Forms
@using Microsoft.AspNetCore.Components.Routing
@using Microsoft.AspNetCore.Components.Web
@using Microsoft.AspNetCore.Components.Web.Virtualization
@using Microsoft.JSInterop
@using Crm.Web
@using Crm.Web.Components
@using Crm.Web.Components.Layout
@using Crm.Web.Components.Shared
@using MudBlazor
@using ApexCharts
```

Bước 4: Cấu hình Components/Layout/MainLayout.razor

Mở src/Crm.Web/Components/Layout/MainLayout.razor và cập nhật các Providers bắt buộc của MudBlazor:

```
@inherits LayoutComponentBase
```

```
@* 3 Providers thiết yếu để MudBlazor hiển thị Theme, Popups, Dialogs và Snackbars *@
```

```
<MudThemeProvider />
```

```
<MudDialogProvider />
```

```
<MudSnackbarProvider />
```

```
<MudPopoverProvider />
```

```
<MudLayout>
```

```
  <MudAppBar Elevation="1" Color="Color.Primary">
```

```
    <MudText Typo="Typo.h6">CRM Enterprise — Day 1 Blazor Server PoC</MudText>
```

```
    <MudSpacer />
```

```
    <MudChip T="string" Color="Color.Secondary" Variant="Variant.Filled" Size="Size.Small">
      Interactive Server
```

```
    </MudChip>
```

```
  </MudAppBar>
```

```
  <MudMainContent>
```

```
    <MudContainer MaxWidth="MaxWidth.Large" Class="mt-6 mb-8">
```

```
      @Body
```

```
        </MudContainer>
    </MudMainContent>
</MudLayout>
```

Lưu ý kiểm tra App.razor: Đảm bảo file src/Crm.Web/Components/App.razor có nạp đầy đủ CSS và JS của MudBlazor trong thẻ <head> và trước thẻ đóng </body>:

```
<link href="https://fonts.googleapis.com/css?family=Roboto:300,400,500,700&display=swap"
rel="stylesheet" />
<link href="_content/MudBlazor/MudBlazor.min.css" rel="stylesheet" />
...
<script src="_framework/blazor.web.js"></script>
<script src="_content/MudBlazor/MudBlazor.min.js"></script>
```

Bước 5: Tạo 3 Component Con trong Components/Shared/

File 1: src/Crm.Web/Components/Shared/MessageComponent.razor

```
@if (!string.IsNullOrEmpty(Message))
{
    <MudAlert Severity="Severity.Success" Variant="Variant.Filled" Dense="true" Class="my-2">
        <strong>Thành công!</strong> @Message
    </MudAlert>
}

@code {
    [Parameter]
    public string? Message { get; set; }
}
```

File 2: src/Crm.Web/Components/Shared/CounterComponent.razor

```
<MudChip T="string" Color="Color.Info" Size="Size.Large"
Icon="@Icons.Material.Filled.TouchApp">
    Đã bấm: <strong>@Count</strong> lần (State in-memory server)
</MudChip>

@code {
    [Parameter]
```

```

    public int Count { get; set; }
}

```

File 3: src/Crm.Web/Components/Shared/SearchComponent.razor

```

@if (Results == null || !Results.Any())
{
    <MudText Color="Color.Default" Typo="Typo.body2" Class="fst-italic pa-2">
        Không tìm thấy kết quả phù hợp.
    </MudText>
}
else
{
    <MudPaper Elevation="0" Outlined="true">
        <MudList T="string" Dense="true">
            @foreach (var item in Results)
            {
                <MudListItem T="string" Icon="@Icons.Material.Filled.Business">
                    @item
                </MudListItem>
            }
        </MudList>
    </MudPaper>
}

@code {
    [Parameter]
    public List<string>? Results { get; set; }
}

```

Bước 6: Tạo trang Demo chính Components/Pages/Demo/Index.razor

Tạo file src/Crm.Web/Components/Pages/Demo/Index.razor:

```

@page "/demo-blazor"
@rendermode InteractiveServer

<PageTitle>Blazor Server Proof of Concept</PageTitle>

<MudText Typo="Typo.h4" Color="Color.Primary" Class="mb-2 font-weight-bold">

```

Blazor Server Proof of Concept — Day 1

</MudText>

<MudText Typo="Typo.subtitle1" Color="Color.Secondary" Class="mb-6">

Kiểm thử cơ chế Interactive Server Component, C# Event Binding, State in-memory và Live Search qua SignalR Persistent Circuit.

</MudText>

@* Test 1: Load Component bằng sự kiện C# *@

<MudPaper Class="pa-6 mb-6" Elevation="3">

<MudText Typo="Typo.h6" Class="mb-2">

<MudIcon Icon="@Icons.Material.Filled.FlashOn" Class="mr-2" />

Test 1: Render Component Con bằng C# Event (Không Reload Trang)

</MudText>

<MudText Typo="Typo.body2" Color="Color.Secondary" Class="mb-3">

Kích hoạt sự kiện OnClick để gán chuỗi trạng thái và render MessageComponent.

</MudText>

<MudButton Variant="Variant.Filled"

Color="Color.Primary"

StartIcon="@Icons.Material.Filled.Download"

OnClick="LoadMessage"

Class="my-2">

Bấm để tải tin nhắn

</MudButton>

<MudPaper Class="pa-4 mt-3" Elevation="0" Style="background: #fafafa; border: 1px dashed #bdbdbd;">

@if (string.IsNullOrEmpty(message))

{

<MudText Color="Color.Default" Typo="Typo.body2">

Chưa có dữ liệu từ server...

</MudText>

}

else

{

<MessageComponent Message="@message" />

}

</MudPaper>

</MudPaper>

@* Test 2: Counter phía server *@

<MudPaper Class="pa-6 mb-6" Elevation="3">

<MudText Typo="Typo.h6" Class="mb-2">

```
<MudIcon Icon="@Icons.Material.Filled.AddCircle" Class="mr-2" />
```

Test 2: Server-side State Management (Persistent Circuit)

```
</MudText>
```

```
<MudText Typo="Typo.body2" Color="Color.Secondary" Class="mb-3">
```

Biến đếm được lưu trên RAM máy chủ, đồng bộ qua WebSocket binary stream.

```
</MudText>
```

```
<MudButton Variant="Variant.Filled"
```

```
Color="Color.Success"
```

```
StartIcon="@Icons.Material.Filled.PlusOne"
```

```
OnClick="IncrementCounter"
```

```
Class="my-2">
```

Bấm để tăng bộ đếm

```
</MudButton>
```

```
<div class="mt-3">
```

```
<CounterComponent Count="@counter" />
```

```
</div>
```

```
</MudPaper>
```

@* Test 3: Live Search với debounce *@

```
<MudPaper Class="pa-6" Elevation="3">
```

```
<MudText Typo="Typo.h6" Class="mb-2">
```

```
<MudIcon Icon="@Icons.Material.Filled.Search" Class="mr-2" />
```

Test 3: Live Search Realtime (Debounce 300ms)

```
</MudText>
```

```
<MudText Typo="Typo.body2" Color="Color.Secondary" Class="mb-3">
```

Nhập liệu tức thì qua MudTextField (Immediate="true"), debounce lọc dữ liệu in-memory.

```
</MudText>
```

```
<MudTextField @bind-Value="searchKeyword"
```

```
@bind-Value:after="OnSearchChanged"
```

```
Placeholder="Nhập từ khóa (vd: Cơ khí, Bulong, Máy)..."
```

```
Variant="Variant.Outlined"
```

```
Immediate="true"
```

```
Clearable="true"
```

```
Adornment="Adornment.Start"
```

```
AdornmentIcon="@Icons.Material.Filled.Search"
```

```
Class="my-2" />
```

```
<div class="mt-3">
```

```
<SearchComponent Results="@searchResults" />
```

```
</div>
```

</MudPaper>

```
@code {
    private string? message;
    private int counter = 0;
    private string? searchKeyword;
    private List<string> searchResults = new();
    private CancellationTokenSource? _searchCts;

    private static readonly List<string> _allItems = new()
    {
        "Nhà máy Cơ khí An Phát",
        "Cơ khí Chính xác Việt Nhật",
        "Khuôn Mẫu Tân Á",
        "Bulong Ốc Vít Nam An",
        "Dụng Cụ Hải Nam",
        "Cơ Khí Thăng Lợi",
        "Chế Tạo Máy Minh Đức",
        "Bơm Công Nghiệp Sài Gòn"
    };

    private void LoadMessage()
    {
        message = $"Dữ liệu phản hồi từ Blazor Circuit lúc {DateTime.Now:HH:mm:ss.fff}";
    }

    private void IncrementCounter()
    {
        counter++;
    }

    private async Task OnSearchChanged()
    {
        // Hủy lượt tìm kiếm trước nếu người dùng tiếp tục gõ phím
        _searchCts?.Cancel();
        _searchCts = new CancellationTokenSource();
        var token = _searchCts.Token;

        try
        {
            // Debounce 300ms chuẩn UI UX
            await Task.Delay(300, token);
        }
    }
}
```

```

        if (string.IsNullOrEmpty(searchKeyword))
        {
            searchResults = new List<string>();
        }
        else
        {
            searchResults = _allItems
                .Where(x => x.Contains(searchKeyword, StringComparison.OrdinalIgnoreCase))
                .ToList();
        }
    }
}
catch (TaskCanceledException)
{
    // Bỏ qua ngoại lệ khi người dùng gõ phím liên tục
}
}
}

```

Bước 7: Trust HTTPS Development Certificate

dotnet dev-certs https --trust

PHASE 5 — CHẠY THỬ NGHIỆM & VERIFY (10 phút)

1. Khởi chạy ứng dụng

```

cd src/Crm.Web
dotnet run

```

Terminal sẽ hiển thị ứng dụng đang lắng nghe kết nối tại:

- <http://localhost:5000>
- <https://localhost:5001>

2. Tiêu chí nghiệm thu (Definition of Done - DoD)

|

| STT | Bước kiểm tra | Thao tác thực hiện | Kết quả kỳ vọng | Status |

| 1 | Trang chủ | Mở trình duyệt vào <https://localhost:5001/> | Trang chủ tải thành công, không phát sinh lỗi 500 hay unhandled exception. |

|

| 2 | Màn hình Demo PoC | Mở <https://localhost:5001/demo-blazor> | Giao diện 3 khối Test hiển thị hoàn chỉnh, font Roboto, theme Material Design của MudBlazor. |

|

| 3 | Load Component Con | Bấm nút "**Bấm để tải tin nhắn**" | Khối thông báo chuyển thành MudAlert xanh hiển thị timestamp, **tuyệt đối không reload trang**. |

|

| 4 | Server-side Counter | Bấm nút "**Bấm để tăng bộ đếm**" 5 lần liên tiếp | MudChip tăng dần số lần bấm **1 → 2 → 3 → 4 → 5**, cập nhật UI tức thì qua SignalR circuit. |

|

| 5 | Live Search Realtime | Gõ từ khóa "cơ khí" vào ô tìm kiếm | Sau **300 ms** debounce, SearchComponent render đúng 3 đơn vị cơ khí tương ứng. |

|

| 6 | Giám sát Circuit WS | Mở **F12** → **Network** → **WS** (WebSockets) | Thấy request kết nối tới `_blazor?id=...` với HTTP Status 101 Switching Protocols giữ trạng thái liên tục. |

|

PHASE 6 — LƯU TRỮ VÀ HOÀN THÀNH NGÀY (5 phút)

1. Khởi tạo & Commit Git

Tại thư mục gốc CrmSolution:

1. Khởi tạo Git

git init

2. Tạo file .gitignore chuẩn cho .NET
dotnet new gitignore

3. Stage và Commit toàn bộ dự án
git add .
git commit -m "feat: setup 4-tier .NET 8 architecture and Blazor Server proof-of-concept"

- Built solution with Domain, Data, Business, Jobs, Web, and Tests projects
- Installed core dependencies: EF Core, Npgsql, QuestPDF, ClosedXML, MailKit, Hangfire, Serilog
- Integrated Blazor Server + MudBlazor 7.x + ApexCharts.Blazor
- Configured interactive server render mode with SignalR WebSocket
- Created /demo-blazor page with 3 interactive tests: load component, counter, live search"

2. Ghi chú nhật ký ngày (notes/daily.md)

Tạo file notes/daily.md:

Daily Log — Day 1 / Sprint 1

Done

- [x] Dựng xong Solution 4-tier + Test project (.NET 8)
- [x] Installed & verified packages cho 5 projects
- [x] Cấu hình Blazor Server + MudBlazor + ApexCharts.Blazor
- [x] Tạo Demo Blazor (/demo-blazor) với 3 test tương tác
- [x] Verify SignalR circuit hoạt động (HTTP 101 WebSocket)
- [x] Commit source code lên Git repository local

Next Steps (Day 2)

- Thiết kế Database Schema SQL cho 15 bảng quản trị CRM
- Cấu hình EF Core DbContext và thiết lập Fluent API
- Tạo Seeder Data thử nghiệm

git add notes/daily.md

git commit -m "docs: add day 1 daily log"

BẢNG TRA CỨU LỖI THƯỜNG GẶP (TROUBLESHOOTING)

| Hiện tượng lỗi | Nguyên nhân gốc rễ | Cách khắc phục chuẩn |

| Lỗi **"Circuit not found"** hoặc ngắt kết nối liên tục | Proxy/Firewall chặn kết nối WebSocket, hoặc cấu hình Nginx thiếu header chuyển mạch giao thức. | Kiểm tra cổng 5001 không bị chặn; trên Nginx reverse proxy bắt buộc có `proxy_set_header Upgrade $http_upgrade;` và `proxy_set_header Connection "upgrade";`. |

| Trang **/demo-blazor** mở lên trắng xóa | Thiếu chỉ thị `@rendermode InteractiveServer` khiến component chỉ render tĩnh (Static SSR). | Thêm directive `@rendermode InteractiveServer` vào đầu file `Index.razor` hoặc cấu hình toàn cục tại `App.razor`. |

| **MudBlazor component hiển thị vỡ layout, mất style** | Chưa khai báo `<MudThemeProvider />` hoặc chưa nhúng file CSS của MudBlazor trong `App.razor`. | Đảm bảo `MainLayout.razor` có đủ `<MudThemeProvider />`, `<MudDialogProvider />`, `<MudSnackbarProvider />`, `<MudPopoverProvider />` và `App.razor` có `CSS_content/MudBlazor/MudBlazor.min.css`. |

| Lỗi biên dịch **"Cannot find Component / Tag name"** | Thiếu namespace chứa các component con trong `_Imports.razor`. | Bổ sung `@using Crm.Web.Components.Shared` và `@using MudBlazor` vào file `src/Crm.Web/Components/_Imports.razor`. |

| **Bấm nút không phản hồi (UI đóng băng)** | Chưa đăng ký Interactive Server Render Mode trong pipeline của `Program.cs`. | Đảm bảo `Program.cs` có `builder.Services.AddRazorComponents().AddInteractiveServerComponents();` và `app.MapRazorComponents<App>().AddInteractiveServerRenderMode();`. |

| **Trình duyệt cảnh báo bảo mật đồ / lỗi HTTPS** | Chứng chỉ phát triển cục bộ (.NET Development Certificate) chưa được Trust. | Chạy lệnh `dotnet dev-certs https --clean` sau đó chạy `dotnet dev-certs https --trust`. |

LƯU Ý ĐẶC THÙ BLAZOR SERVER

1. Vòng đời Circuit (Persistent Connection):

Mỗi người dùng kết nối sẽ duy trì một circuit (WebSocket) lưu trữ component state in-memory trên RAM máy chủ. Khi mạng gián đoạn, client tự động thử kết nối lại; nếu ngắt quá **30 – 60** giây, circuit bị giải phóng và trang cần reload.

2. Quy tắc gọi `StateHasChanged()`:

Trong các sự kiện do người dùng kích hoạt trực tiếp từ component (`OnClick`, `OnInput`), Blazor Server tự động kích hoạt `StateHasChanged()`. Chỉ cần gọi phương thức này thủ công khi cập nhật trạng thái từ background threads, Timer, hoặc sự kiện bên ngoài luồng UI.

3. Chiến lược JS Interop:

Tuyệt đối ưu tiên sử dụng **100%** C# và các component có sẵn của MudBlazor. Chỉ sử dụng JavaScript Interoperability (IJSRuntime) cho các tác vụ trình duyệt chuyên biệt (như SortableJS trên Kanban Board hoặc ApexCharts wrappers).

4. **Kỹ thuật Debugging:**

Đặt breakpoint trực tiếp trong Visual Studio / VS Code tương tự Console App hoặc Web API vì toàn bộ mã nguồn Razor Component được thực thi trực tiếp trên server runtime.

5. **Tiêu chuẩn Tài nguyên & Hiệu năng:**

Mỗi active circuit tiêu thụ trung bình $\approx 100 - 200 \text{ KB}$ RAM. Với quy mô $10 - 20$ nhân viên nội bộ cùng thao tác, hệ thống chỉ chiếm thêm $\approx 2 - 4 \text{ MB}$ RAM, máy chủ VPS **2 GB** vận hành ổn định và dư thừa tài nguyên.